

```
-- =====
-- AGUTS – SQL SELECT Queries (Final Submission)
-- IT-214 DBMS I Lab Group 5 I DA-IICT I Winter 2025-26
-- Ahmedabad Gandhinagar Unified Transit System
-- =====
-- Group Members:
-- Hari Sharma      : 202401195
-- Arjunsinh Vaghela : 202401235
-- Tirth Ditani     : 202401417
-- Neeti Gunsai     : 202401423
-- Rudra Bhatt      : 202401461
-- =====
```

SET search_path TO aguts;

```
-- =====
-- SCENARIO A: NETWORK & INFRASTRUCTURE
-- Real-world use: Passenger journey planner, admin route
-- management, city transport planning dashboard
-- =====
```

-- Q1. Stops serving more than one route (transfer points).

-- Used by: Intermodal journey planning

```
SELECT s.stop_name, s.stop_type,
       COUNT(DISTINCT rs.route_id) AS route_count,
       STRING_AGG(r.route_number, ','
                  ORDER BY r.route_number) AS routes_served
FROM   Stop s
JOIN   Route_Stop rs ON rs.stop_id = s.stop_id
JOIN   Route r      ON r.route_id = rs.route_id
GROUP BY s.stop_id, s.stop_name, s.stop_type
HAVING COUNT(DISTINCT rs.route_id) > 1
ORDER BY route_count DESC;
```

-- Q2. All depots with operator, city, and address.

-- Used by: Depot management, vehicle dispatch

```
SELECT d.depot_id, d.depot_name, d.address,
       o.operator_name, c.city_name
FROM   Depot d
JOIN   Transit_Operator o ON o.operator_id = d.operator_id
JOIN   City c            ON c.city_id    = d.city_id
ORDER BY o.operator_name, d.depot_name;
```

-- Q3. Total network length and route count per operator.

-- Used by: Annual report, city transport statistics

```
SELECT o.operator_name,
       COUNT(r.route_id)      AS total_routes,
       SUM(r.distance_km)     AS total_network_km,
       ROUND(AVG(r.distance_km),2) AS avg_route_km
FROM   Transit_Operator o
```

```

JOIN Route r ON r.operator_id = o.operator_id
GROUP BY o.operator_id, o.operator_name
ORDER BY total_network_km DESC;

```

-- Q4. Geographic coordinates of all stops (for map display).

-- Used by: Passenger app map, route visualization

```

SELECT s.stop_id, s.stop_name, s.stop_type,
       s.latitude, s.longitude, c.city_name,
       o.operator_name
FROM Stop s
JOIN City c ON c.city_id = s.city_id
JOIN Route_Stop rs ON rs.stop_id = s.stop_id
JOIN Route r ON r.route_id = rs.route_id
JOIN Transit_Operator o
      ON o.operator_id = r.operator_id
GROUP BY s.stop_id, s.stop_name, s.stop_type,
         s.latitude, s.longitude, c.city_name,
         o.operator_id, o.operator_name
ORDER BY o.operator_name, s.stop_name;

```

```

-- =====
-- SCENARIO B: SCHEDULE & TRIP OPERATIONS
-- Real-world use: Operations control centre, driver app,
-- delay management dashboard
-- =====

```

-- Q1. On-time performance (OTP) percentage per operator.

-- Used by: KPI dashboard, management reports

```

SELECT o.operator_name,
       COUNT(t.trip_id) AS total_trips,
       SUM(CASE WHEN t.trip_status='Completed'
                THEN 1 ELSE 0 END) AS on_time,
       SUM(CASE WHEN t.trip_status='Delayed'
                THEN 1 ELSE 0 END) AS delayed,
       SUM(CASE WHEN t.trip_status='Cancelled'
                THEN 1 ELSE 0 END) AS cancelled,
       ROUND(100.0 *
            SUM(CASE WHEN t.trip_status='Completed'
                    THEN 1 ELSE 0 END)
            / COUNT(t.trip_id), 1) AS otp_percent
FROM Trip t
JOIN Route r ON r.route_id = t.route_id
JOIN Transit_Operator o ON o.operator_id = r.operator_id
GROUP BY o.operator_id, o.operator_name
ORDER BY otp_percent DESC;

```

-- Q2. Routes with the most delayed trips.

-- Used by: Route performance review, bottleneck detection

```

SELECT r.route_number, r.route_name, o.operator_name,
       COUNT(t.trip_id) AS delayed_trips
FROM   Trip t
JOIN   Route r ON r.route_id = t.route_id
JOIN   Transit_Operator o ON o.operator_id = r.operator_id
WHERE  t.trip_status = 'Delayed'
GROUP BY r.route_id, r.route_number,
         r.route_name, o.operator_name
ORDER BY delayed_trips DESC;

```

-- Q3. Daily trip count trend across Jan-Mar 2024.

-- Used by: Operations trend analysis, monthly report

```

SELECT t.trip_date,
       COUNT(t.trip_id) AS daily_trips,
       SUM(CASE WHEN t.trip_status='Completed'
                THEN 1 ELSE 0 END) AS completed,
       SUM(CASE WHEN t.trip_status='Delayed'
                THEN 1 ELSE 0 END) AS delayed,
       SUM(CASE WHEN t.trip_status='Cancelled'
                THEN 1 ELSE 0 END) AS cancelled
FROM   Trip t
WHERE  t.trip_date BETWEEN '2024-01-01' AND '2024-03-31'
GROUP BY t.trip_date
ORDER BY t.trip_date;

```

```

-- =====
-- SCENARIO C: PASSENGER & SMART CARD MANAGEMENT
-- Real-world use: Passenger app, customer care portal,
-- smart card management system (like GMRC Go Card)
-- =====

```

-- Q1. Passengers with low smart card balance (below Rs.50).

-- Used by: Low-balance alert system, auto-recharge trigger

```

SELECT p.passenger_id,
       p.first_name || ' ' || p.last_name AS passenger_name,
       p.email, pp.phone,
       sc.card_id, sc.balance,
       sc.card_status, sc.expiry_date
FROM   Smart_Card sc
JOIN   Passenger p ON p.passenger_id = sc.passenger_id
LEFT JOIN Passenger_Phone pp
      ON pp.passenger_id = p.passenger_id
      AND pp.phone_type = 'Mobile'
WHERE  sc.balance < 50
      AND sc.card_status NOT IN ('Blocked','Expired')
ORDER BY sc.balance;

```

-- Q2. Passenger registration growth by month.

-- Used by: User acquisition tracking (like real apps)

```

SELECT TO_CHAR(sc.issue_date, 'YYYY-MM') AS month,
       COUNT(sc.card_id) AS new_cards_issued,
       COUNT(DISTINCT sc.passenger_id) AS new_passengers
FROM   Smart_Card sc
GROUP BY TO_CHAR(sc.issue_date, 'YYYY-MM')
ORDER BY month;

```

```

-- Q3. Top 10 most frequent travellers by transaction count.
--   Used by: Loyalty programme, frequent rider recognition

```

```

SELECT p.first_name || ' ' || p.last_name AS passenger,
       p.concession_type,
       COUNT(tt.txn_id) AS total_transactions,
       SUM(tt.amount) AS total_spent_inr
FROM   Passenger p
JOIN   Ticket_Transaction tt
      ON tt.passenger_id = p.passenger_id
GROUP BY p.passenger_id, p.first_name,
         p.last_name, p.concession_type
ORDER BY total_transactions DESC
LIMIT 10;

```

```

-- =====
-- SCENARIO D: TICKETING & REVENUE
-- Real-world use: Finance dashboard, ticketing audit,
-- pass management system
-- =====

```

```

-- Q1. Concession-wise fare comparison: GMRC vs BRTS.
--   Used by: Policy comparison, subsidy calculation

```

```

SELECT o.operator_name, f.concession_type,
       MIN(f.fare_amount) AS min_fare,
       MAX(f.fare_amount) AS max_fare,
       ROUND(AVG(f.fare_amount),2) AS avg_fare
FROM   Fare f
JOIN   Transit_Operator o ON o.operator_id = f.operator_id
WHERE  o.operator_name IN ('GMRC','Janmarg BRTS')
GROUP BY o.operator_id, o.operator_name, f.concession_type
ORDER BY o.operator_name, f.concession_type;

```

```

-- Q2. Revenue generated per route.

```

```

--   Used by: Route profitability analysis
SELECT r.route_number, r.route_name, o.operator_name,
       COUNT(tt.txn_id) AS transactions,
       SUM(tt.amount) AS route_revenue_inr
FROM   Ticket_Transaction tt
JOIN   Trip t ON t.trip_id = tt.trip_id
JOIN   Route r ON r.route_id = t.route_id
JOIN   Transit_Operator o ON o.operator_id = r.operator_id

```

```

GROUP BY r.route_id, r.route_number,
        r.route_name, o.operator_name
ORDER BY route_revenue_inr DESC;

```

-- Q3. Each operator's percentage share of total revenue.

-- Used by: Finance — revenue attribution, cost sharing

```

SELECT o.operator_name,
       SUM(tt.amount) AS operator_revenue,
       ROUND(100.0 * SUM(tt.amount)
             / (SELECT SUM(amount) FROM Ticket_Transaction),
             2) AS revenue_share_pct
FROM   Ticket_Transaction tt
JOIN   Trip t ON t.trip_id = tt.trip_id
JOIN   Route r ON r.route_id = t.route_id
JOIN   Transit_Operator o ON o.operator_id = r.operator_id
GROUP BY o.operator_id, o.operator_name
ORDER BY operator_revenue DESC;

```

```

-- =====
-- SCENARIO E: FLEET & MAINTENANCE
-- Real-world use: Depot manager system, maintenance scheduler,
-- vehicle health monitoring dashboard
-- =====

```

-- Q1. Full maintenance history per vehicle with duration.

-- Used by: Maintenance log, service history

```

SELECT v.registration_no, o.operator_name, d.depot_name,
       m.maintenance_type, m.description,
       m.start_date, m.end_date,
       (m.end_date - m.start_date + 1) AS duration_days
FROM   Maintenance m
JOIN   Vehicle v ON v.vehicle_id = m.vehicle_id
JOIN   Transit_Operator o ON o.operator_id = v.operator_id
JOIN   Depot d ON d.depot_id = v.depot_id
ORDER BY m.start_date DESC;

```

-- Q2. Fleet breakdown: trains vs buses per operator.

-- Used by: Fleet overview dashboard

```

SELECT o.operator_name,
       CASE WHEN mt.vehicle_id IS NOT NULL
            THEN 'Metro Train' ELSE 'Bus'
       END AS vehicle_type,
       COUNT(v.vehicle_id) AS fleet_count,
       SUM(v.capacity) AS total_capacity,
       ROUND(AVG(v.capacity),0) AS avg_capacity
FROM   Vehicle v
JOIN   Transit_Operator o ON o.operator_id = v.operator_id
LEFT JOIN Metro_Train mt ON mt.vehicle_id = v.vehicle_id
LEFT JOIN Bus b ON b.vehicle_id = v.vehicle_id
GROUP BY o.operator_id, o.operator_name, vehicle_type

```

```
ORDER BY o.operator_name, vehicle_type;
```

```
-- Q3. Vehicles not used in any trip for 60+ days (idle risk).
```

```
-- Used by: Idle asset detection, depot cost review
```

```
SELECT v.registration_no, o.operator_name,  
       d.depot_name, v.status, v.manufacture_year,  
       MAX(t.trip_date) AS last_trip_date,  
       (CURRENT_DATE - MAX(t.trip_date)) AS days_idle  
FROM   Vehicle v  
JOIN   Transit_Operator o ON o.operator_id = v.operator_id  
JOIN   Depot d           ON d.depot_id   = v.depot_id  
LEFT JOIN Trip t         ON t.vehicle_id = v.vehicle_id  
GROUP BY v.vehicle_id, v.registration_no,  
         o.operator_name, d.depot_name,  
         v.status, v.manufacture_year  
HAVING MAX(t.trip_date) IS NULL  
       OR (CURRENT_DATE - MAX(t.trip_date)) >= 60  
ORDER BY days_idle DESC NULLS FIRST;
```

```
-- =====  
-- SCENARIO F: STAFF & OPERATIONS  
-- Real-world use: HR system, depot manager portal,  
-- driver/conductor duty management  
-- =====
```

```
-- Q1.. Full staff listing with role, operator, and depot.
```

```
-- Used by: HR directory, staff management system
```

```
SELECT st.staff_id,  
       st.first_name || ' ' ||  
       COALESCE(st.middle_name || ' ', '') ||  
       st.last_name   AS full_name,  
       st.role, o.operator_name,  
       d.depot_name, st.date_of_joining, st.contact  
FROM   Staff st  
JOIN   Transit_Operator o ON o.operator_id = st.operator_id  
JOIN   Depot d           ON d.depot_id   = st.depot_id  
ORDER BY o.operator_name, st.role, st.first_name;
```

```
-- Q2. All complaints with status, route, and action flag.
```

```
-- Used by: Admin complaint dashboard (like GMRC portal)
```

```
SELECT c.complaint_id,  
       p.first_name || ' ' || p.last_name AS passenger,  
       c.category, c.description, c.status,  
       c.file_date, c.resolved_date,  
       r.route_name, o.operator_name,  
       CASE WHEN c.status='Open'  
            THEN 'Action Needed'  
            WHEN c.status='In Progress'  
            THEN 'Being Handled'
```

```

        ELSE 'Closed'
    END AS action_flag
FROM Complaint c
JOIN Passenger p ON p.passenger_id = c.passenger_id
JOIN Trip t ON t.trip_id = c.trip_id
JOIN Route r ON r.route_id = t.route_id
JOIN Transit_Operator o ON o.operator_id = r.operator_id
ORDER BY
    CASE c.status
        WHEN 'Open' THEN 1
        WHEN 'In Progress' THEN 2
        ELSE 3
    END,
    c.filed_date;

```

```

-- Q3. Staff certifications expiring within 180 days.
-- Used by: Compliance alert, renewal scheduling
SELECT st.first_name || ' ' || st.last_name AS staff_name,
       st.role, o.operator_name,
       sc.certification,
       sc.issued_date, sc.expiry_date,
       (sc.expiry_date - CURRENT_DATE) AS days_to_expiry
FROM Staff st
JOIN Staff_Certification sc ON sc.staff_id = st.staff_id
JOIN Transit_Operator o ON o.operator_id = st.operator_id
WHERE sc.expiry_date BETWEEN CURRENT_DATE
      AND CURRENT_DATE + INTERVAL '180 days'
ORDER BY days_to_expiry;

```